

AquaLLM: Adaptive Large Language Model Agents for Autonomous Aquaculture Management

Phu Leon Hoang
Tickle College of Engineering
University of Tennessee, Knoxville
Knoxville, TN
phoang5@vols.utk.edu

Abstract—This report investigates the application of Large Language Models (LLMs) as autonomous decision-making agents for managing a biologically-inspired aquaculture simulation. The novelty of this work lies in adapting memory-driven reflection and tool-augmented reasoning from LLM agents to the complex, stochastic domain of environmental management. The proposed system, AquaLLM, utilizes a cognitive architecture where the agent observes pond states, reflects on past actions, and leverages external tools to optimize policies for Largemouth Bass (*Micropterus salmoides*). By balancing survival rates, fish health, and strict budgetary constraints over multi-generational timelines, the agent successfully navigates the exploration-exploitation tradeoff. Ablation studies demonstrate that memory, planning, and tool-use are critical for preventing ecological collapse and maintaining economic viability in chaotic simulated environments.

Index Terms—Large Language Models, Embodied Agents, Agent-Based Modeling, Aquaculture, Particle Swarm Optimization

I. INTRODUCTION

A. Project Goal

In modern aquaculture, managing the delicate balance between fish health, environmental stability, and economic viability remains a high-stakes challenge. Poor management can trigger catastrophic biological collapses, such as ammonia toxicity or pathogen outbreaks, which are often exacerbated by the complex swarm behaviors of fish populations [?]. The overarching goal of this project is to develop an autonomous agent, AquaLLM, that acts as a virtual farmer. This agent is tasked with managing resource distribution—specifically oxygen, food, and probiotics—for Largemouth Bass within a chaotic 3D simulation while adhering to a rigorous \$500 budget.

B. Research Question

This paper investigates whether an embodied Large Language Model (LLM) can effectively navigate the stochastic feedback loops of a biological system. Specifically, we examine the extent to which cognitive scaffolding, such as external tool usage and generational memory, allows an LLM to replace traditional mathematical optimizers. We further explore the trade-off between model scale and computational efficiency, questioning if the performance gains of a 72B parameter model justify the resource expenditure compared to a leaner 7B model.

C. Topic Significance and Solution Architecture

Effective aquaculture management is often hindered by the unpredictable nature of fish shoal behavior and delayed environmental feedback [?]. By leveraging recent advancements in generative agents [?] and embodied AI [?], AquaLLM introduces a cognitive architecture designed to handle these complexities. Our solution integrates semantic environmental observations with long-term memory compaction, enabling the agent to learn from past failures and optimize resource allocation across multiple generations.

II. PREVIOUS WORK

Agent-based modeling (ABM) is a standard method for understanding complex biological systems, allowing researchers to simulate factors like dissolved oxygen and fish shoal behavior [?]. In the specific context of Largemouth Bass aquaculture, researchers have documented the severe environmental impacts of poor pond management on survival rates [?].

Traditionally, optimization within these ABMs relies on heuristic algorithms. This project utilizes recent advancements in generative agents as a foundational reference. Park et al. (2023) demonstrated the efficacy of agents that maintain memory streams to simulate believable interactive behavior [?]. We adapt this memory-driven reflection for environmental management, allowing AquaLLM to record past days and adjust policies. Furthermore, our architecture draws inspiration from Voyager, an embodied agent that mastered complex virtual worlds through iterative tool-use and feedback [?].

III. TECHNICAL APPROACH

A. Problem Domain and Rules

The environment is a 3D simulated pond ($180 \times 85 \times 50$ units) containing Largemouth Bass. The simulation is evaluated at 1-hour timesteps over 30 days. The agent must maximize a Fitness score while staying under a \$500 budget.

B. Core Simulation Engine Implementation

Entities and Physics: The environment manages dynamic resources and biological waste. A strict eutrophication cycle is implemented: uneaten food decays into pollutants, which transform into hazards like ammonia clouds and disease zones.
Agent-Based Behavior: Fish move via a modified Particle

Swarm Optimization (PSO) algorithm, modeling swarm intelligence to mimic realistic foraging and hazard avoidance.

C. Cognitive LLM Agent Architecture

AquaLLM operates as a continuous loop. The system is compartmentalized into four modules, governed by templates in Appendix C:

1) *Observation*: Discretizes the pond into a localized grid and computes a “warning points” metric. The high-level objectives and action boundaries for this module are established by the `_CORE_TEMPLATE` (Appendix C-1).

2) *Tool*: Integrates Python functions like `calculate_budget` to anchor reasoning in mathematical reality, as constrained by the `_TOOLS_BLOCK` (Appendix C-4).

3) *Planning*: Provides Chain-of-Thought reasoning to formulate rationales. This is scaffolded by the `_PLANNING_BLOCK` and `_COUNTER_BLOCK` (Appendix C-3).

4) *Memory*: Records daily policies and observations. Historical context is compacted to remain within token limits, serving the `_REFLECTION_USAGE_BLOCK` (Appendix C-2).

D. Premium Cognitive Features

Multi-Tiered Reflection: Combines Memory and Planning (Appendix C-2) to establish better policies across generations. *Early Quit*: Requires Memory, Tool, and Planning modules. As dictated by the `_EARLY_QUIT_BLOCK` (Appendix C-6), the agent can abort doomed runs starting from Generation 3.

E. Examined Options (Ablation)

Tested configurations include *Full*, *No Planning*, *No Memory*, *No Tools*, *No Observer*, and *Minimal*.

IV. DATASET AND IMPLEMENTATION

A. Generative Dataset Domain

AquaLLM operates as an embodied agent where the 3D physical simulation acts as a live data engine, providing dynamic interaction outcomes between the fish and the agent’s decisions.

B. Hardware and Execution Environment

The project uses Google Colab Pro for Students with an NVIDIA H100 GPU to accelerate physics updates and local LLM generations.

C. LLM Configuration and Context Limits

The framework uses Qwen2.5 (7B and 72B). A 30-day simulation demands up to 500,000 tokens, which overflows the context limits of both models. The Memory Compaction module is used to prevent fatal crashes.

D. Robustness and Syntax Enforcement

Fallback enforcement includes Levenshtein fuzzy matching and an iterative retry loop (up to 2 attempts).

V. EXPERIMENTS AND RESULTS ANALYSIS

A. Experimental Setup

Experiments were conducted over 5 generations with 50 fish per pond. The primary metric is Fitness:

$$\text{Fitness} = 0.55 \cdot \text{Yield} + 0.33 \cdot \text{Saving} + 0.12 \cdot \text{Health} \quad (1)$$

B. Cross-Model Overall Performance Comparison

Figure ?? compares the ultimate Fitness achieved. While the 72B model outperforms the 7B model, the delta is not particularly impressive given the higher computational cost.

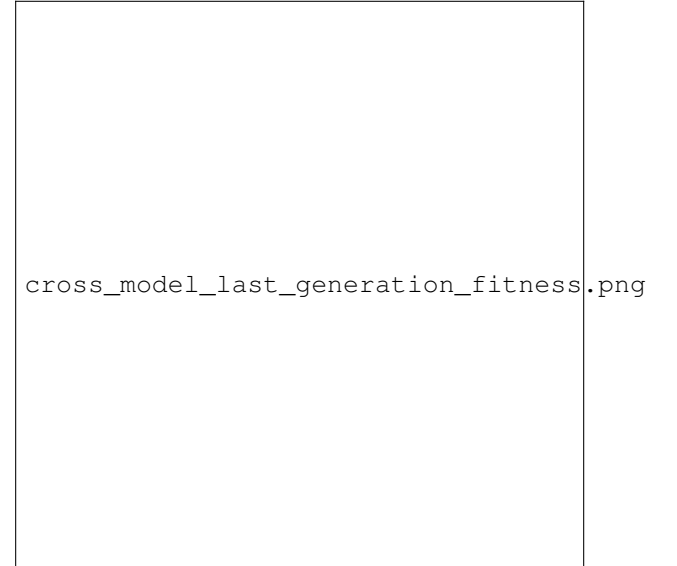


Fig. 1. Final Generation Fitness Comparison (Qwen2.5 7B vs 72B).

C. Learning Curve: Fitness by Generation

Figures ?? and ?? show the *no_memory* configuration exhibits a severe drop, as amnesic agents fail to stabilize. The 72B model achieves faster stabilization.

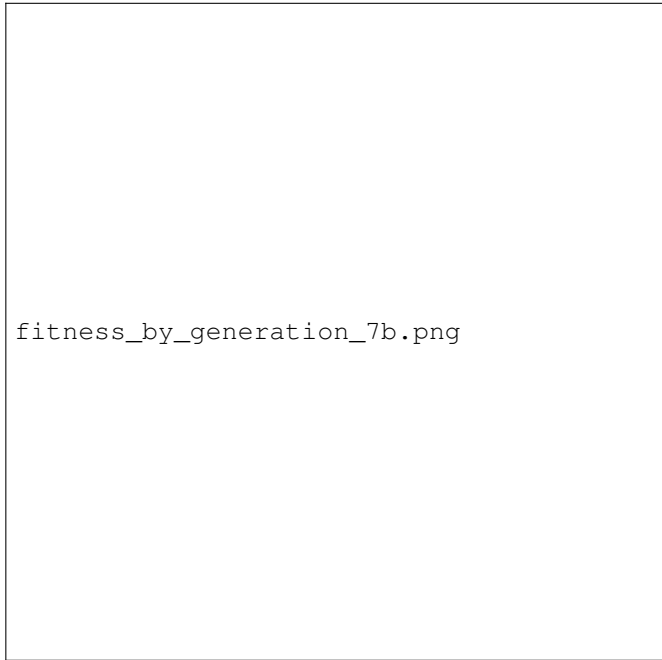


Fig. 2. Fitness by Generation (7B)



Fig. 4. Final Generation Metrics (7B)

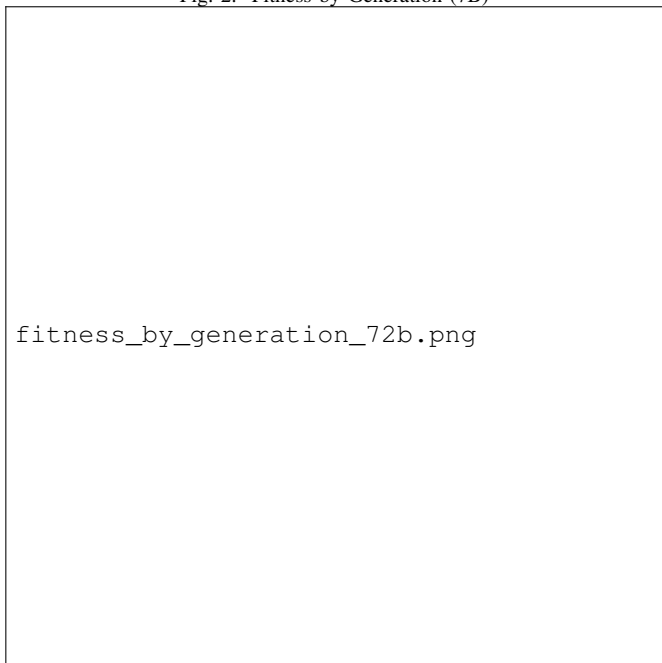


Fig. 3. Fitness by Generation (72B)

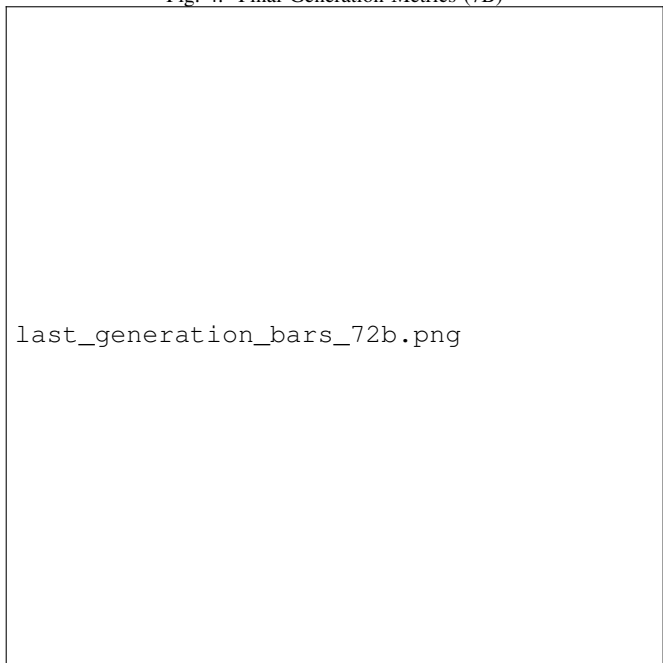


Fig. 5. Final Generation Metrics (72B)

D. Final Generation Metrics

Figures ?? and ?? indicate the 72B model’s high spending results in healthier fish. Notably, *minimal* outperforms *no_memory*, suggesting the shorter context in *minimal* contains less noise. This confirms memory is the critical factor.

E. Importance Heatmap

The heatmaps (Figures ?? and ??) show the 7B model’s fragility under *No Memory*. The 72B model suffers under *No Tools*, confirming scale cannot replace arithmetic grounding.



Fig. 6. Ablation Heatmap (7B)

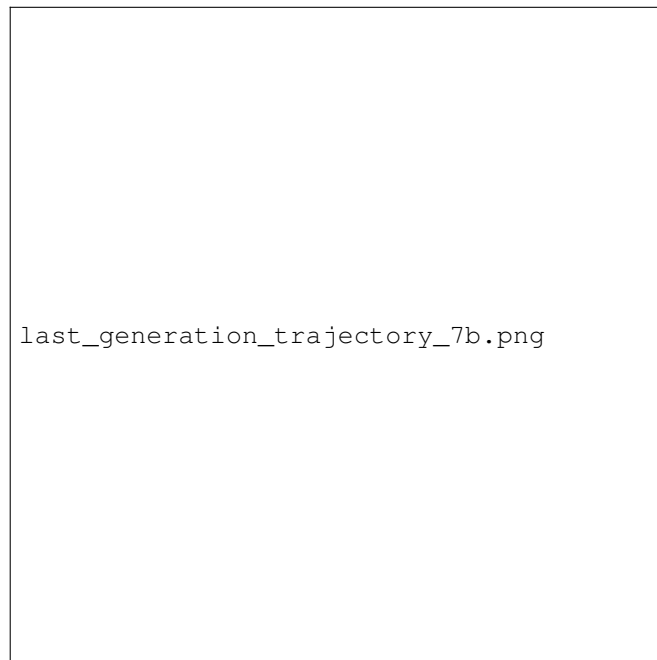


Fig. 8. 30-Day Trajectory (7B)

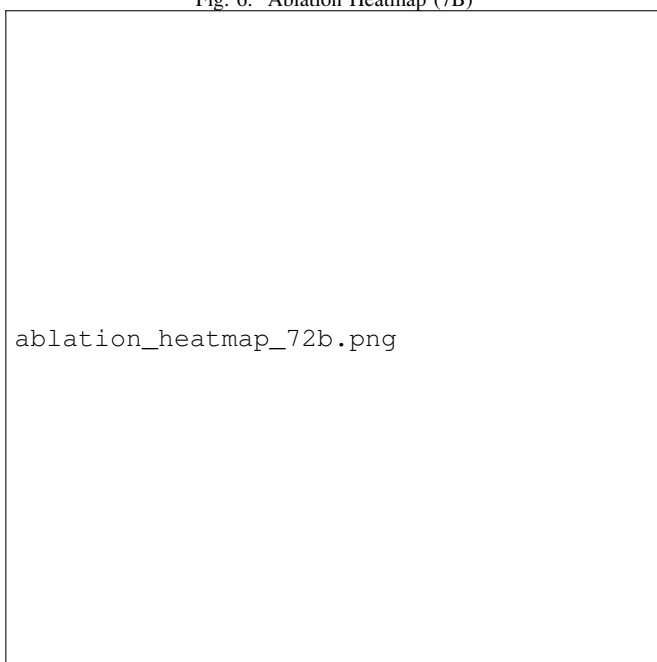


Fig. 7. Ablation Heatmap (72B)

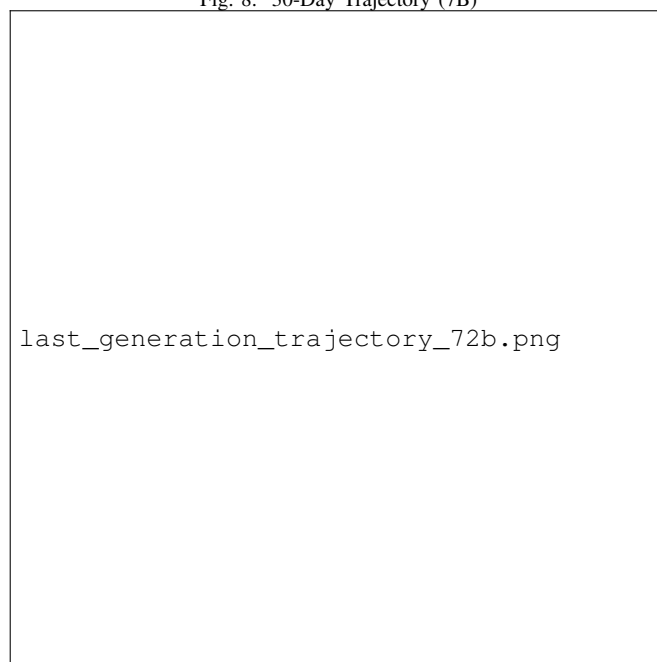


Fig. 9. 30-Day Trajectory (72B)

F. Resource Management Trajectory

Trajectories (Figures ?? and ??) show both models perform equally under the *Full* architecture, maintaining controlled depletion.

G. Computational Overhead: Token and Word Usage

Figures ?? and ?? show raw total context. Both models fundamentally overflow context windows without compaction.

token_word_usage_7b.png

Fig. 10. Token Usage (7B)

token_word_usage_72b.png

Fig. 11. Token Usage (72B)

VI. CONCLUSION

This study demonstrated that Large Language Models can act as effective autonomous agents in complex biological simulations. Our results confirm that while a larger model with an expanded context window improves reasoning and multi-objective optimization, the performance gains do not currently justify the exponentially higher computational cost. Furthermore, our experiments were constrained by low generational cycles and single-run executions due to hardware limitations. Given the highly stochastic and chaotic nature of

the virtual pond, we conclude that an even larger model scale, or perhaps more specialized training on biological causality, is required to fully master the role of a virtual farmer. Ultimately, memory compaction and tool-augmentation remain the most critical architectural requirements for embodied agents in high-dimensional resource management tasks.

VII. APPENDICES

A. Code Design

The codebase is separated into two primary functional domains:

- *Core Simulation (/core)*: Logic for entities, physics, and behaviors.
- *Agent Architecture (/agent)*: Logic for memory, observation, and tools.

The implementation of the core simulation environment and the development of several cognitive modules—specifically the initial drafting of the system prompts and the logic for the Memory Compaction feature—were significantly accelerated through the use of Claude Code. This AI-assisted development workflow allowed for rapid iteration between the stochastic biological requirements of the simulation and the technical constraints of the LLM agent architecture.

B. Workload Distribution

I was the sole contributor to this project. All tasks, including simulation physics design, LLM agent architecture development, prompt engineering, execution of experiments, and empirical analysis, were completed individually.

C. System Prompt Engineering

1. Overarching Objectives (_CORE_TEMPLATE)

You manage an aquaculture fish pond. {days}d \${budget} budget. {fish} fish fixed.

There are {gens} generations total. Use previous generation results only as evidence for better decisions; do not experiment casually. The last generation is graded and is very important.

Best strategy: Gen-1 Day-0 should start with the cheapest valid conservative policy, then later generations should improve from evidence. A cheap baseline reveals what actually fails without wasting budget early.

This is a losing game: Fitness usually declines over time. Goal: prevent sharp Fitness drops while preserving Saving.

$Fitness = 0.55 * Yield + 0.33 * Saving + 0.12 * Health$. Saving is important. Over budget/all dead=Fitness 0.

Initial policy is extremely important. Prefer a conservative, affordable initial policy unless prior evidence clearly supports spending more.

Cost rule: smaller interval = more frequent = higher cost. Larger interval = cheaper. Smaller quantity/duration = cheaper. Larger quantity/duration = higher cost.

Valid ranges: food_interval 1-12, food_quantity 5-30, probiotic_interval 24-168, probiotic_quantity 1-4, oxygen_interval 1-20, oxygen_duration 1-4, locations 0-9.

Never recommend invalid values like probiotic_interval=12. Avoid food_interval=10 fixation.

Reflections are token-capped. Keep every reflection short: one concise lesson plus at most one param=value.

Only fish stats: health, fullness, immunity, oxygen. ALL intervals are HOURS. Output compact JSON only.

Locations: 0=Center 1=TL 2=TR 3=BL 4=BR 5=TC 6=BC 7=LC 8=RC 9=Random.

2. Reflection Protocol (*_REFLECTION_USAGE_BLOCK*)

Reflection types have different purposes and all reflections are token-capped, so keep them short:

- CHECKPOINT reflection: mid-generation review; use only for next-day adjustment in the current generation.
- WARNING reflection: urgent mid-generation warning; use only for next-day adjustment in the current generation. A warning often means spending/feeding/pumping too much is creating waste, pollutants, NH₃, disease, parasites, or budget pressure. For WARNING reflections, first consider spending LESS: increase intervals, reduce quantity/duration, or move drops away from hazards. Do not automatically spend more unless the warning clearly says fish are starving or suffocating.
- END_GEN_REFLECTION: generation summary; use only to choose the next generation Day-0 initial policy.
- EARLY_QUIT_REFLECTION: failed-start summary; use only to avoid that bad initial policy on the restarted/next generation.

Do not confuse these. Mid-gen reflections guide tomorrow. End-gen and Early-Quit reflections guide the next initial policy. Reflection format should be short: reason; optional NEXT_START: param=value.

3. Chain-of-Thought Scaffolding (*_PLANNING_BLOCK & _COUNTER_BLOCK*)

Think briefly: Fitness drop mild or sharp? weakest stat? cheapest fix? budget? Which reflection type is present, and should it affect tomorrow or next generation Day-0? If WARNING reflection is present, ask whether overspending/pollution is the cause before spending more. Is candidate identical to CURRENT_POLICY? If identical, keep policy. JSON.

If reflection says exact param=value, use it only if valid, affordable, different from current policy, and appropriate for that reflection type.

Rules: mild decline + stats OK -> keep policy and save. sharp drop/stat<0.5 -> cheapest fix. WARNING reflection -> first suspect overspending, waste, pollutants, hazards, or budget pressure; spending less is often better. low_fullness->adjust food carefully; leftover food causes pollution. low_oxygen->oxygen_interval first, duration only if severe. low_immunity->probiotic sparingly; probiotic_interval cannot be below 24. hazards->move drops away/reduce food. over budget->increase oxygen_interval, reduce oxygen_duration, then increase food_interval.

4. Mathematical Grounding (*_TOOLS_BLOCK*)

Cost/day: food=(24/food_interval)*food_quantity*\$0.05; prob=(24/probiotic_interval)*probiotic_quantity*\$0.75; O2=(24/oxygen_interval)*oxygen_duration*\$2.50. Smaller interval costs more. Smaller quantity/duration costs less. Target average \${daily:.2f}/day. Use calculate_budget when changing.

5. Temporal Action Execution (*_FIRST_ROUND_BLOCK & _ADJUST_BLOCK*)

Day0: initial policy matters most. If this is Gen-1, start with the cheapest valid conservative baseline unless it is obviously impossible. If this is Gen-2 or later, use END_GEN_REFLECTION and EARLY_QUIT_REFLECTION as evidence for the initial policy. Do not use CHECKPOINT/WARNING reflections as next-generation initial-policy rules unless the end-gen reflection repeats them. Do not default food_interval=10. Use prior BEST as baseline if shown. Output ONLY compact JSON with 9 policy fields, no fish_count.

Read observation and CURRENT_POLICY. Use CHECKPOINT/WARNING reflections for next-day adjustment only. For WARNING, prefer spending less unless fish

are clearly starving/suffocating. Ignore END_GEN_REFLECTION/EARLY_QUIT_REFLECTION for same-day emergency changes unless they are restated as current evidence. If decline is mild, keep CURRENT_POLICY to preserve Saving. If sharp drop/stat<0.5, change the cheapest effective parameter. If candidate JSON equals CURRENT_POLICY, that is no change required. Output ONLY compact JSON: 9 fields + early_quit. No fish_count.

6. Proactive Abort Trigger (*_EARLY_QUIT_BLOCK*)

Early-Quit rule: Starting from Gen 3, if restart attempts remain, you may Early-Quit only after calling compare_day_fitness. Early-Quit is allowed only when current day Fitness is at least 5% below the average Fitness of the same day in previous generations. If Early-Quit is justified, output JSON with early_quit=true and keep policy fields unchanged.

REFERENCES

- [1] P. Urbanova et al., “Agent Based Modeling of Fish Shoal Behavior,” *LNCS*, pp. 3–13, 2023.
- [2] H. Hou et al., “An Environmental Impact Assessment of Largemouth Bass Aquaculture,” *Sustainability*, 2023.
- [3] J. S. Park et al., “Generative Agents: Interactive Simulacra of Human Behavior,” *Proc. UIST '23*, 2023.
- [4] G. Wang et al., “Voyager: An Open-Ended Embodied Agent with Large Language Models,” *arXiv:2305.16291*, 2023.